

Создание и вставка элементов на JavaScript

Сейчас мы научимся создавать новые DOM элементы через JavaScript, а затем добавлять их на страницу. Для этого предназначен метод `createElement`. Параметром этого метода следует передавать имя тега, который должен быть создан.

Если записать результат работы `createElement` в переменную, то в этой переменной будет такой элемент, как будто бы мы получили его через `querySelector`.

Единственное отличие - наш элемент не будет размещен на странице. А так мы можем менять ему текст, атрибуты, навешивать обработчики событий и в конце концов разместить его на странице.

Для размещения нового элемента на странице применяется метод `appendChild`. Этот метод следует применять к тому элементу, в который мы хотим поместить наш элемент. А параметром метода следует передавать наш новый элемент, созданный ранее через `createElement`.

Посмотрим на практическом примере. Пусть у нас есть див, а в нем - несколько абзацев:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5 </div>
```

Давайте создадим абзац, установим ему текст и поместим на страницу в конец блока `#parent`:

```
1 let parent = document.querySelector('#parent');
2
3 let p = document.createElement('p');
4 p.textContent = '!';
5
6 parent.appendChild(p);
```

Результат выполнения кода:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5   <p>!</p>
6 </div>
```

№ 1

©jsPmMnCIE

Дан ol:

```
1 <ol id="elem"></ol>
```

Вставьте ему в конец li с текстом 'item'.

№ 2

©jsPmMnCIE

Дан ol и кнопка:

```
1 <ol id="elem"></ol>
2 <button id="button">click me</button>
```

Сделайте так, чтобы по клику на кнопку в конец списка добавлялся li с текстом 'item'.

Привязывание событий при вставке элементов

Давайте теперь при вставке элементов будем привязывать к ним обработчики событий.

Пусть, к примеру, у нас есть вот такой родительский элемент:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5 </div>
```

Давайте добавим в конец этого родителя еще один абзац, установив ему обработчик клика:

```
1 let parent = document.querySelector('#parent');
2
3 let p = document.createElement('p');
4 p.textContent = '!';
5
6 p.addEventListener('click', function() {
7   console.log(this.textContent); // по клику выведем текст абзаца
8 });
9
10 parent.appendChild(p);
```

№ 1

©jsPmMnIBE

Дан `ol` и кнопка. Сделайте так, чтобы по клику на кнопку в конец `ol` добавлялся тег `li`. Сделайте так, чтобы по клику на любой из добавленных `li` в конец его текста записывался восклицательный знак.

Создание элементов в цикле на JavaScript

В предыдущих уроках мы создавали по одному новому элементу. Давайте теперь сделаем так, чтобы новые элементы создавались в цикле.

Пусть для примера у нас есть вот такой див-родитель:

```
1 <div id="parent"></div>
```

Давайте запустим цикл, который добавит в конец нашего дива 9 новых абзацев:

```
1 let parent = document.querySelector('#parent');
2
3 for (let i = 1; i <= 9; i++) {
4   let p = document.createElement('p');
5   p.textContent = '!';
6
7   parent.appendChild(p);
8 }
```

№ 1

©jsPmMnLEC

Дан пустой `ul`. Запустите цикл, который вставит в него 10 тегов `li`. Сделайте так, чтобы текстом каждой `li` был ее порядковый номер.

Навешивание обработчиков в цикле на JavaScript

В предыдущем уроке мы научились создавать новые элементы в цикле. Давайте теперь будем навешивать обработчики событий при создании новых элементов.

Пусть у нас опять есть вот такой див-родитель:

```
1 | <div id="parent"></div>
```

Запустим цикл, который добавит в конец нашего дива 9 новых абзацев, навесив на них обработчик клика:

```
1 | let parent = document.querySelector('#parent');
2 |
3 | for (let i = 1; i <= 9; i++) {
4 |   let p = document.createElement('p');
5 |   p.textContent = '!';
6 |
7 |   // Навешиваем обработчик клика:
8 |   p.addEventListener('click', function() {
9 |     console.log(this.textContent);
10 |   });
11 |
12 |   parent.appendChild(p);
13 | }
```

№ 1

©jsPmMnLBH

Дан див. Запустите цикл, который добавит в наш див 5 инпутов. Пусть дан также абзац. Сделайте так, чтобы каждый из новых инпутов по потере фокуса записывал свой текст в абзац.

Удаление элементов на JavaScript

Давайте теперь научимся удалять элементы. Для этого нужно использовать метод `remove`. Посмотрим на примере, как это делается. Пусть у нас есть абзацы:

```
1 | <p>elem 1</p>
2 | <p>elem 2</p>
3 | <p>elem 3</p>
4 | <p>elem 4</p>
5 | <p>elem 5</p>
```

Давайте сделаем так, чтобы любой абзац удалялся по клику на нем:

```
1 | let elems = document.querySelectorAll('p');
2 |
3 | for (let elem of elems) {
4 |   elem.addEventListener('click', function() {
5 |     elem.remove();
6 |   });
7 | }
```

№ 1

©jsPmMnER

Дан следующий код:

```
1 | <ul>
2 |   <li>1</li>
3 |   <li>2</li>
4 |   <li>3</li>
5 | </ul>
```

Сделайте так, чтобы любая `li` удалялась по клику на нее.

Дан следующий код:

```
1 <ul id="parent">
2   <li>1</li>
3   <li>2</li>
4   <li>3</li>
5 </ul>
6
7 <input type="submit" id="button">
```

Сделайте так, чтобы по каждому клику на кнопку удалялся последний элемент из #parent.

Вставка элементов в начало или в конец на JavaScript

Изучите теорию по следующим ссылкам:

[append](#) [prepend](#)

Метод append

Метод `append` позволяет вставить в конец какого-либо элемента другой элемент. Параметром метод принимает элемент, как правило созданный через `createElement`, либо строку. Можно добавить сразу несколько элементов или строк, перечислив их через запятую.

Синтаксис

```
1 родитель.append(элемент или строка);
```

Пример

Давайте создадим абзац, установим ему текст и поместим на страницу в конец блока #parent:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5 </div>

1 let parent = document.querySelector('#parent');
2
3 let p = document.createElement('p');
4 p.textContent = '!';
5
6 parent.append(p);
```

Результат выполнения кода:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5   <p>!</p>
6 </div>
```

Метод prepend

Метод `prepend` позволяет вставить в начало какого-либо элемента другой элемент. Параметром метод принимает элемент, как правило созданный через `createElement`, либо строку. Можно добавить сразу несколько элементов или строк, перечислив их через запятую.

Синтаксис

```
1 | родитель.prepend(элемент или строка);
```

Пример

Давайте создадим абзац, установим ему текст и поместим на страницу в начало блока #parent:

```
1 | <div id="parent">
2 |   <p>1</p>
3 |   <p>2</p>
4 |   <p>3</p>
5 | </div>

1 | let parent = document.querySelector('#parent');
2 |
3 | let p = document.createElement('p');
4 | p.textContent = '!';
5 |
6 | parent.prepend(p);
```

Результат выполнения кода:

```
1 | <div id="parent">
2 |   <p>!</p>
3 |   <p>1</p>
4 |   <p>2</p>
5 |   <p>3</p>
6 | </div>
```

№ 1

©jsPmMnIE

Дан ul:

```
1 | <ul id="elem">
2 |   <li>text</li>
3 |   <li>text</li>
4 |   <li>text</li>
5 |   <li>text</li>
6 |   <li>text</li>
7 | </ul>
```

Вставьте ему в начало элемент с текстом 'start', а в конец - элемент с текстом 'finish'.

Вставка элементов перед элементами на JavaScript

Изучите теорию по следующим ссылкам:

[insertBefore](#)

Метод insertBefore

Метод `insertBefore` позволяет вставить элемент перед другим элементом. Чаще всего используется после создания элемента с помощью `createElement`. Метод применяется к родителю того элемента, перед которым произойдет вставка.

Синтаксис

```
1 | родитель.insertBefore(элемент, перед кем вставить);
```

Пример

Создадим абзац и поместим его перед вторым абзацем:

```
1 <div id="parent">
2   <p>elem 1</p>
3   <p id="before">elem 2</p>
4   <p>elem 3</p>
5 </div>

1 let parent = document.querySelector('#parent');
2 let before = document.querySelector('#before');
3
4 let p = document.createElement('p');
5 p.textContent = '!';
6
7 parent.insertBefore(p, before);
```

Результат выполнения кода:

```
1 <div id="parent">
2   <p>elem 1</p>
3   <p>!</p>
4   <p>elem 2</p>
5   <p>elem 3</p>
6 </div>
```

№ 1

Дан элемент #elem:

```
1 <ul id="parent">
2   <li>text</li>
3   <li>text</li>
4   <li id="elem">text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Вставьте перед этим элементом элемент с текстом 'new'.

Смежная вставка элементов на JavaScript

Изучите теорию по следующим ссылкам:

[insertAdjacentElement](#)

Метод insertAdjacentElement

Метод `insertAdjacentElement` позволяет вставить элемент в любое место страницы. Чаще всего используется после создания элемента с помощью `createElement`. Код вставляется относительно *опорного элемента*. Можно сделать вставку перед опорным элементом (способ вставки `beforeBegin`), после него (способ вставки `afterEnd`), а также в начало (способ вставки `afterBegin`) или в конец (способ вставки `beforeEnd`) опорного элемента.

Синтаксис

```
1 опорный элемент.insertAdjacentElement(способ вставки, код для вставки);
```

Пример . Способ beforeBegin

Пусть опорный элемент - это элемент #target. Вставим перед ним новый абзац:

```
1 <div id="target">
2   <p>elem</p>
3 </div>
```

```
1 let p = document.createElement('p');
2 p.textContent = '!!!';
3
4 let target = document.querySelector('#target');
5 target.insertAdjacentElement('beforeBegin', p);
```

Результат выполнения кода:

```
1 <p>!</p>
2 <div id="target">
3   <p>elem</p>
4 </div>
```

Пример . Способ afterEnd

Пример . Способ afterBegin

Пример . Способ beforeEnd

№1

©jsPmMnIA

Дан элемент:

```
1 <div id="elem" style="border: 1px solid red;">
2   <p>text</p>
3   <p>text</p>
4   <p>text</p>
5 </div>
```

Вставьте перед ним абзац с текстом '!!!'.

№2

©jsPmMnIA

Дан элемент:

```
1 <div id="elem" style="border: 1px solid red;">
2   <p>text</p>
3   <p>text</p>
4   <p>text</p>
5 </div>
```

Вставьте после него абзац с текстом '!!!'.

Дан элемент:

```
1 <div id="elem" style="border: 1px solid red;">
2   <p>text</p>
3   <p>text</p>
4   <p>text</p>
5 </div>
```

Вставьте ему в начало абзац с текстом '!!!'.

Дан элемент:

```
1 <div id="elem" style="border: 1px solid red;">
2   <p>text</p>
3   <p>text</p>
4   <p>text</p>
5 </div>
```

Вставьте ему в конец абзац с текстом '!!!'.

Смежная вставка тегов на JavaScript

Изучите теорию по следующим ссылкам:

[insertAdjacentHTML](#)

Метод insertAdjacentHTML

Метод insertAdjacentHTML позволяет вставить строку HTML кода в любое место страницы. Код вставляется относительно *опорного элемента*. Можно сделать вставку перед опорным элементом (способ вставки 'beforeBegin'), после него (способ вставки 'afterEnd'), а также в начало (способ вставки 'afterBegin') или в конец (способ вставки 'beforeEnd') опорного элемента.

Синтаксис

```
1 | опорный элемент.insertAdjacentHTML(способ вставки, код для вставки);
```

Пример . Способ beforeBegin

Пусть опорный элемент - это элемент #target. Вставим перед ним новый абзац:

```
1 <div id="target">
2   <p>elem</p>
3 </div>

1 let target = document.querySelector('#target');
2 target.insertAdjacentHTML('beforeBegin', '<p>!</p>');
```

Результат выполнения кода:

```
1 <p>!</p>
2 <div id="target">
3   <p>elem</p>
4 </div>
```


Пример . Способ afterEnd

Пример . Способ afterBegin

Пример . Способ beforeEnd

№ 1

©jsPmMnIAT

Дан элемент:

```
1 <div id="elem" style="border: 1px solid red;">
2   <p>text</p>
3   <p>text</p>
4   <p>text</p>
5 </div>
```

Дан элемент. Вставьте перед ним следующий тег:

```
1 <div class="www"><p>text</p><p>text</p><input></div>
```

Клонирование элементов на JavaScript

В данном уроке мы научимся получать копии элементов. С этими копиями можно будет работать, как с обычными элементами - изменять их и вставлять в нужное место страницы. Процесс получения копий элементов называется *клонирование*.

Клонировать элемент можно с помощью метода `cloneNode`. В этот метод нужно передавать параметром `true` либо `false`. Если передан `true`, то элемент копируется полностью, вместе со всем атрибутами и дочерними элементами, а если `false` - только сам элемент.

Давайте посмотрим на примере. Пусть у нас есть вот такой код:

```
1 <div id="parent">
2   <div class="elem">
3     <p>первый абзац</p>
4     <p>второй абзац</p>
5   </div>
6 </div>
```

Сделаем копию блока с классом `elem` и вставим его в конец блока `#parent`:

```
1 let parent = document.querySelector('#parent');
2 let elem = parent.querySelector('.elem');
3
4 let clone = elem.cloneNode(true);
5 parent.appendChild(clone);
```

В результате получится следующее:

```
1 <div id="parent">
2   <div class="elem">
3     <p>первый абзац</p>
4     <p>второй абзац</p>
5   </div>
6   <div class="elem">
7     <p>первый абзац</p>
8     <p>второй абзац</p>
9   </div>
10 </div>
```

Дан инпут. Дана кнопка. По нажатию на кнопку клонируйте этот инпут.

Проверка элементов в JavaScript

Изучите теорию по следующим ссылкам:

[matches](#) [contains](#)

Метод matches

Метод `matches` позволяет проверить, удовлетворяет ли элемент указанному CSS селектору.

Синтаксис

```
1 | элемент.matches('селектор');
```

Пример

Проверим, является ли наш элемент абзацем с классом `www`:

```
1 | <p id="elem" class="www"></p>

1 | let elem = document.querySelector('#elem');
2 | console.log(elem.matches('p.www'));
```

Результат выполнения кода:

```
1 | true
```

Метод contains

Метод `contains` позволяет проверить, содержит ли один элемент внутри себя другой. Параметром метода передается элемент, который будет проверяться на то, что он находится внутри элемента, к которому применялся метод.

Синтаксис

```
1 | родитель.contains(элемент);
```

Пример

Проверим, находится ли абзац `#child` в блоке `#parent`:

```
1 | <div id="parent">
2 |   <p id="child"></p>
3 | </div>

1 | let parent = document.querySelector('#parent');
2 | let child = document.querySelector('#child');
3 |
4 | let contains = parent.contains(child);
5 | console.log(contains);
```

Результат выполнения кода:

```
1 | true
```

№ 1

©jsPmMnECh

Дан элемент:

```
1 | <div id="elem" class="www"></div>
```

Проверьте, является ли этот элемент элементом с классом `www`.

№ 2

©jsPmMnECh

Дан элемент. Проверьте, является ли этот элемент абзацем.

№ 3

©jsPmMnECh

Даны две переменные `elem1` и `elem2`, содержащие два элемента:

```
1 | <div id="elem1">
2 |   <p id="elem2"></p>
3 | </div>
```

```
1 | let elem1 = document.querySelector('#elem1');
2 | let elem2 = document.querySelector('#elem2');
```

Проверьте, является ли элемент из `elem2` потомком элемента из `elem1`.